AI Health Report & Medicine Analyzer

Project Report

Course	Artificial Intelligence
Instructor	Riaz Ahmed
Team Member 1	Rameen Ramzan — 24K-0557
Team Member 2	Umama Zubair — 24K-0621

1. Introduction

Medical lab reports are one of the most common yet least understood documents a patient receives. Numbers appear next to reference ranges, values are flagged as high or low, yet the patient leaves the clinic with no clear explanation of what any of it means. The result is unnecessary panic, misguided self-treatment, or complete dismissal of results that actually require attention.

The AI Health Report & Medicine Analyzer addresses this gap directly. It is a web-based application that accepts a lab report PDF, automatically extracts all test values, classifies each result as Normal, High, or Low against standard medical reference ranges, and then uses Google Gemini AI to produce a plain-language explanation of the findings. The system also includes a dedicated medicine interaction checker, allowing users to enter a list of their current medications and receive an AI-generated safety analysis of any risky combinations.

The application was built with accessibility as the primary design goal: no medical background should be required to understand the output, and no value should be presented without context. All AI-generated content includes a disclaimer directing users to consult a qualified physician before acting on any result.

1.1 Motivation

Two problems motivated this project. First, patients routinely receive lab reports they cannot interpret. Even when a test result is flagged abnormal, there is no mechanism in a standard printed report to explain the clinical significance of that deviation in everyday language. Second, polypharmacy — the use of multiple medications simultaneously, is increasingly common, yet accessible tools for checking drug interactions remain technical and intimidating for the average user.

By combining automatic PDF parsing, rule-based classification, and a large language model, this project creates a single accessible interface that addresses both problems without requiring any medical expertise from the user.

1.2 Objectives

- Allow users to upload a standard lab report PDF and automatically extract all test values without manual entry.
- Classify each extracted value as Normal, High, or Low using a reference range database covering 50+ common tests.
- Use the Gemini API to generate a clear, warm, plain-language explanation of the results, highlighting what abnormal values may indicate.
- Provide an overall risk assessment (Low / Moderate / High) based on which tests are abnormal and their clinical weight.
- Build a medicine interaction checker where users enter drug names and receive an AI-generated safety analysis.

2. System Overview

The application is structured as a two-tab Streamlit interface. The first tab handles lab report analysis; the second handles medicine interaction checking. Both tabs share a common Gemini

AI backend. The overall data flow is linear: the user uploads a file or enters text, the system processes it, and the results are displayed on the same page with no navigation required.

2.1 High-Level Architecture

The system is composed of five distinct processing layers that operate in sequence:

Layer	Module	Responsibility
Input	app.py	File upload, gender selection, user interaction
Extraction	pdf_parser.py	PDF text and table extraction, name cleaning
Classification	classifier.py + reference_range.py	Value-to-range comparison, status tagging
Risk Scoring	risk_engine.py	Weighted risk score, overall risk level
AI Generation	gemini_client.py	Gemini API calls, plain-language summaries
Drug Safety	medicine_checker.py	Multi-drug interaction analysis via Gemini

2.2 Workflow

The following steps describe the complete processing pipeline for a lab report upload:

1. User uploads a PDF lab report and selects their gender (for gender-specific reference ranges).
2. pdf_parser.py extracts all text and structured tables from the PDF using pdfplumber.
3. Each detected test name is cleaned and matched against a database of aliases and reference ranges.
4. classifier.py tags each value as Normal, High, or Low and builds a structured DataFrame.
5. risk_engine.py calculates an overall risk level using weighted scores for clinically significant tests.
6. The results are displayed as colour-coded cards or a table. An AI Explanation can be generated on demand via Gemini.
7. A follow-up chat interface allows the user to ask specific questions about their results.

3. Module Descriptions

3.1 PDF Parser (pdf_parser.py)

The PDF parser is responsible for converting an uploaded lab report PDF into structured data. It uses pdfplumber to extract both raw text and structured tables from every page of the document. Two parsing strategies run in parallel and their results are merged:

- Table parsing: extracts rows from detected PDF tables, reading the first cell as the test name and scanning subsequent cells for a numeric value and a reference range (e.g. 4.5 – 11.0).
- Text parsing: applies a regex pattern anchored to the start of each line to capture short test name tokens followed immediately by a numeric value. This catches reports that are not structured as formal tables.

A critical function, clean_test_name(), was developed to handle the common problem of PDF cells containing long descriptive text alongside the test name. For example, a cell reading "Egfr >90 Ml/Min/1.73 M2 Interpretation: -------------- Some Loss Of Renal Function..." is reduced to simply "Egfr" through four sequential steps: splitting at colons, splitting at dashes, regex-trimming at the first standalone number, and capping at 40 characters.

Results are filtered against a keyword whitelist of valid medical tests and deduplicated by normalised name before being returned.

3.2 Classifier (classifier.py + reference_range.py + aliases.py)

Once raw values are extracted, each one is classified by comparing it against a reference range database. The database (reference_range.py) contains minimum and maximum values for over 50 common blood tests, with separate entries for male, female, and general populations where clinically relevant.

An alias dictionary (aliases.py) maps the many names a single test can appear under to a canonical key. For example, 'hb', 'hgb', and 'haemoglobin' all resolve to 'Hemoglobin'. Name matching first attempts a direct case-insensitive lookup, then falls back to alias scanning.

The output of this module is a Pandas DataFrame with columns for Test Name, Value, Unit, Status, Min, and Max, which serves as the single source of truth for all downstream components.

3.3 Risk Engine (risk_engine.py)

The risk engine calculates an overall risk score from the classified results DataFrame. Each abnormal test contributes a weighted score to a running total based on its clinical significance. High-weight tests include Troponin I (weight 10), BNP (8), CK-MB (8), Glucose and HbA1c (6 each), and Creatinine and eGFR (6 each). Tests not in the weight table contribute a default weight of 2.

The final risk level is determined by a combination of the total score and the percentage of abnormal results. A score of 15 or more, or 40% or more abnormal results, triggers a High risk classification. A score between 6 and 14 or 20–39% abnormal results triggers Moderate. All other outcomes are Low.

3.4 Gemini Client (gemini_client.py)

All AI-generated content passes through gemini_client.py, which wraps the Google Generative AI SDK. The module exposes three functions:

- explain_lab_results(df): Formats the results DataFrame into a compact text summary and sends it to Gemini with a structured prompt requesting a 200–300 word plain-language explanation. The prompt instructs the model to describe what each abnormal test measures, what the value may indicate, and to group all normal tests into a single reassuring sentence.
- answer_followup(question, df, history): Supports a multi-turn chat session by sending the full conversation history, the results data, and the new question together in a single prompt. The model is instructed to answer in 3–5 simple sentences without making a diagnosis.
- check_medicine_interactions(meds_list): Sends a list of drug names to Gemini with a prompt requesting an interaction risk level, a short explanation, and precaution advice.

All calls include a retry loop with a 2-second backoff and return a user-friendly error message if the service remains unavailable after three attempts.

3.5 Medicine Checker (medicine_checker.py)

The medicine checker accepts a plain-text list of drug names entered by the user (minimum two). The names are passed directly to Gemini via a safety-focused prompt that asks for an interaction risk rating (Low / Moderate / High), a brief explanation of any known interactions, and specific precaution advice. The response is rendered as formatted markdown in the UI. A disclaimer is appended both in the prompt and in the interface caption.

3.6 Medical Insights (medical_insights.py)

This module provides short, rule-based one-line insights for common test results without requiring an API call. It covers haematological markers (Hemoglobin, RBC, WBC, Platelets), metabolic markers (Glucose, HbA1c), liver enzymes (ALT, AST, Bilirubin), kidney markers (Creatinine, Urea/BUN), and thyroid function (TSH). These insights are displayed alongside each result card as an immediate, always-available interpretation.

4. User Interface

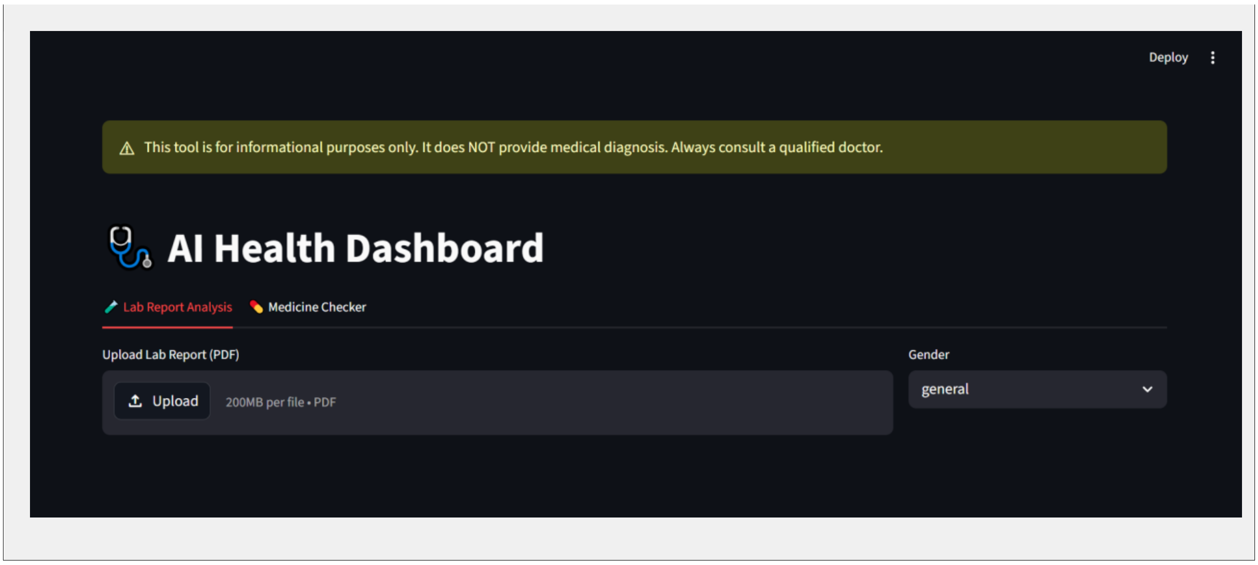
The interface is built entirely in Streamlit and is organised into two tabs accessible from the top of the page. A global disclaimer banner appears at the top of every page reminding users that the tool is for informational purposes only.

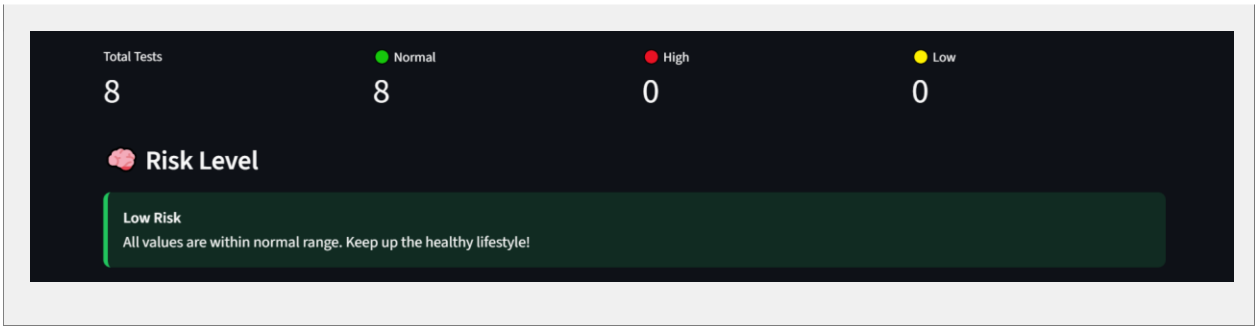
4.1 Lab Report Analysis Tab

This tab contains the complete lab report processing pipeline. The user uploads a PDF and selects their gender, then the following sections are rendered in sequence after processing:

- Summary Metrics: Four metric cards display the total number of tests, and the count of Normal, High, and Low results at a glance.
- Risk Level: A colour-coded banner (green for Low, amber for Moderate, red for High) displays the overall risk level and a plain-language explanation of the score.

- Results View: A radio toggle allows the user to switch between a card layout and a table layout. Cards are colour-coded by status using semi-transparent backgrounds compatible with both light and dark mode. The table shows only the five most useful columns.
- AI Explanation: A button triggers an on-demand Gemini API call that returns a structured plain-language summary of all results. The output is rendered as readable markdown paragraphs.
- Chat Interface: A persistent chat input allows the user to ask follow-up questions. The full conversation history is rendered using Streamlit's native chat_message components.





📋 Results

View as

○ Cards ◉ Table

	Test Name	Value	Unit	Status	Min	Max
0	Blood Urea Nitrogen	11	-	Normal	5	18
1	Serum Creatinine	0.7	-	Normal	0.6	1.1
2	Egfr	90	-	Normal	60	999
3	Serum Direct Bilirubin	0.2	-	Normal	0	0.2
4	S.Indirect Bilirubin	0.2	-	Normal	0.1	0.8
5	Sgpt (Alt)	13	-	Normal	7	45
6	Sgot (Ast)	21	-	Normal	10	40
7	Serum Total Bilirubin	0.4	mg/dl	Normal	0.1	1.2

◉ Cards ○ Table

Blood Urea Nitrogen 🟢
11.0 -
Status: **Normal**

Serum Creatinine 🟢
0.7 -
Status: **Normal**

Egfr 🟢
90.0 -
Status: **Normal**

Serum Direct Bilirubin 🟢
0.2 -
Status: **Normal**

S.Indirect Bilirubin 🟢
0.2 -
Status: **Normal**

Sgpt (Alt) 🟢
13.0 -
Status: **Normal**

Sgot (Ast) 🟢
21.0 -
Status: **Normal**

Serum Total Bilirubin 🟢
0.4 mg/dl
Status: **Normal**

AI Explanation

Generate AI Report

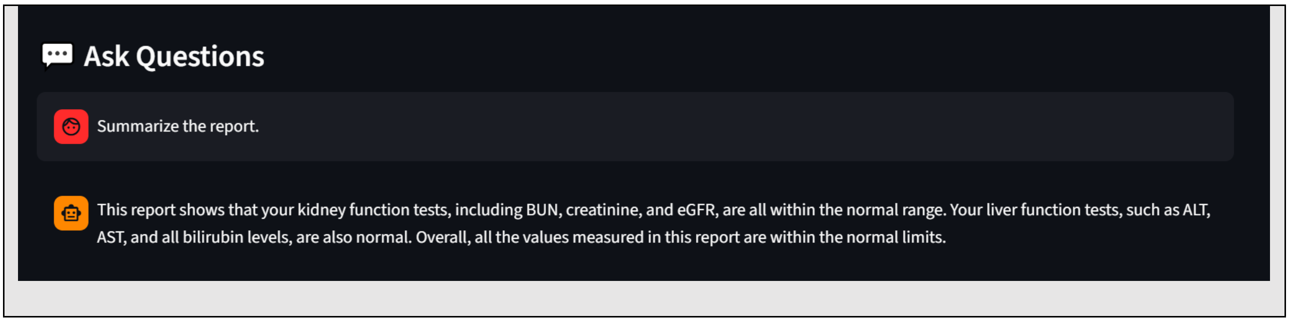
Hi there! I've had a good look at your recent lab report, and I'm delighted to share that all your results appear very healthy and comfortably within the normal ranges. This is excellent news!

Let's go through them. We checked several important markers related to your kidney health. Your Blood Urea Nitrogen, Serum Creatinine, and eGFR (which is a calculation giving us a good estimate of how well your kidneys are filtering waste from your blood) all came back perfectly normal. This is truly wonderful news, suggesting your kidneys are functioning very efficiently and healthily.

We also looked closely at your liver health with a set of tests. Your Sgpt (ALT) and Sgot (AST) are enzymes that, when elevated, can sometimes signal liver stress or damage, but yours are both well within the healthy limits. Similarly, your Serum Direct Bilirubin, S. Indirect Bilirubin, and Serum Total Bilirubin, which help us assess how your liver processes waste products, are all comfortably in the normal range. So, these results collectively tell us your liver is doing a fantastic job too.

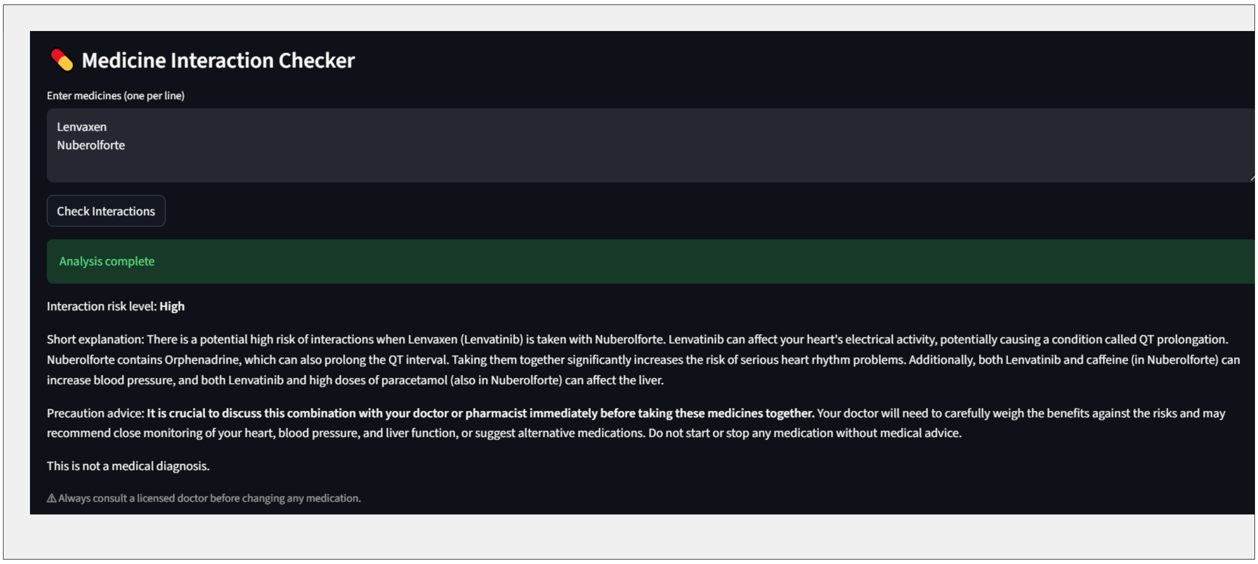
In summary, every single test on your report looks healthy and exactly as we'd hope to see them. It truly indicates a positive picture for these aspects of your health.

While I hope this explanation helps you understand your results better, please remember it's always important to discuss these over with your own doctor. They can put these results into context with your unique medical history and offer personalized advice.



4.2 Medicine Checker Tab

The second tab presents a text area where users enter one medicine name per line. Clicking the Check Interactions button sends the list to Gemini and displays the formatted interaction analysis. A minimum of two medicines is required. A caption below the button reinforces the disclaimer.



5. AI Design and Prompt Engineering

The quality of the AI output depends entirely on prompt design. Several deliberate choices were made to ensure responses are safe, readable, and appropriately scoped.

5.1 Lab Explanation Prompt

The explanation prompt structures the expected output into four explicit sections: an overall summary sentence, a per-abnormal-test paragraph explaining what the test measures and what the value may suggest, a grouped sentence for all normal results, and a closing reminder to consult a doctor. The prompt prohibits tables, JSON, markdown headers, and clinical jargon, and constrains the total word count to between 200 and 300 words. The tone instruction 'warm and reassuring, not alarming' was added after early tests produced responses that unnecessarily alarmed users.

5.2 Follow-up Chat Prompt

Each follow-up query includes the full conversation history, the complete formatted results table, and the new question in a single prompt. This stateless approach ensures the model always has full context without requiring a persistent session. Answers are constrained to 3–5 sentences and the model is explicitly told not to offer a diagnosis.

5.3 Medicine Interaction Prompt

The medicine interaction prompt asks the model to act as a medical safety assistant and return three specific items: a risk level (Low / Moderate / High), a short explanation of the interaction mechanism, and practical precaution advice. The phrase 'This is not a medical diagnosis' is required to appear in the output by the prompt itself, ensuring the disclaimer is always present regardless of how Gemini structures its response.

6. Testing and Validation

6.1 PDF Parsing

The parser was tested against multiple real lab report PDFs with different layouts: table-based reports, plain-text reports, and mixed-format reports. The clean_test_name() function was developed specifically in response to observed failures where long descriptive cell content was being captured as the test name. After the fix, test names are consistently short and clean across all tested formats.

6.2 Classification Accuracy

Classification was validated by manually comparing the application output against the reference ranges printed on the same lab reports. Results matched in all cases for tests present in the reference range database. Tests not present in the database are assigned an 'Unknown' status and excluded from risk scoring.

6.3 AI Response Quality

AI explanations were reviewed for clarity, accuracy of framing, and appropriate scope. The responses consistently describe what a test measures and what an abnormal value may suggest without making a diagnosis. The tone is consistently non-alarming. Follow-up chat responses remain on-topic and within the 3–5 sentence constraint.

7. Libraries and Dependencies

The following libraries are required to run the application:

Library	Version	Purpose
streamlit	≥ 1.32.0	Web interface, layout, widgets, and session state management
google-generativeai	≥ 0.5.0	Google Gemini API client for all AI-generated content
pdfplumber	≥ 0.10.0	PDF text extraction and table parsing

pandas	≥ 2.0.0	Structured DataFrame for lab results
matplotlib	≥ 3.7.0	Retained as dependency for compatibility with existing imports
python-dotenv	≥ 1.0.0	Loading the GEMINI_API_KEY from a local .env file

10. Conclusion

The AI Health Report & Medicine Analyzer successfully demonstrates how a combination of PDF text extraction, rule-based classification, and large language model generation can be composed into a practical, accessible health information tool. The application processes a lab report PDF end-to-end from raw upload to plain-language AI explanation — in a single browser session, with no technical knowledge required from the user.

The project addressed several real engineering challenges: cleaning noisy PDF content into compact test names, building a reference range system with alias resolution and gender-specific ranges, designing a risk scoring engine with clinically weighted inputs, and engineering prompts that produce safe and readable AI output. The dark mode UI fix and the chat history implementation further improved the robustness and usability of the application.

While the current system is intentionally limited to informational use and always directs users to consult a physician, it demonstrates meaningful potential as an assistive layer between raw lab data and patient understanding — a gap that is currently underserved by existing tools.